ETHANS X
CLOUD & AI ACADEMY

Data Handling with

Python (pandas)

In the first chapter, you were introduced to the pandas library and learned how to load data into a DataFrame. You saw what a DataFrame looks like and how to take a first glance at its contents. That was just the beginning.

In this chapter, we go deeper into the practical skills that data analysts use every single day. Real-world data is rarely clean or complete. It contains missing values, incorrect formats, duplicate entries, and columns that need to be reshaped before any analysis can begin. The seven topics covered in this chapter build on each other in a logical sequence — from simply picking the right rows and columns, all the way to computing meaningful summaries from groups of data.

We will work with a consistent sample dataset throughout the entire chapter — an Employee Dataset representing a small company's HR records. Using the same dataset across all topics allows you to clearly see how each operation transforms the data step by step.

What You Will Learn in This Chapter
1. Data Selection and Filtering — Picking exactly the rows and columns you need
2. Handling Missing Values — Detecting and dealing with gaps in your data
3. Data Cleaning and Transformation — Fixing, reshaping, and standardising data
4. Sorting and Ranking Data — Ordering data by values and assigning ranks
5. Grouping and Aggregation (groupby()) — Summarising data by categories
6. Merging and Joining DataFrames — Combining multiple tables intelligently
7. Basic Data Analysis and Summary Statistics — Computing key numerical insights

The Dataset We Will Use

Throughout this chapter, all code examples and explanations use the Employee Dataset shown below. It represents records of eight employees across different departments. Notice that some values are intentionally missing (shown as NaN) — we will learn to handle those in Section 2.

emp_id	name	department	salary	experience	city
101	Ananya Sharma	HR	48000	3	Mumbai
102	Rahul Verma	Engineering	75000	6	Bengaluru
103	Priya Nair	Finance	62000	5	Chennai
104	Karan Mehta	Engineering	80000	8	Bengaluru
105	Sneha Pillai	HR	NaN	2	Mumbai
106	Vikram Das	Finance	58000	4	Delhi
107	Meera Joshi	Engineering	72000	NaN	Pune
108	Arjun Kapoor	HR	51000	3	Mumbai

Table 2.0 — Employee Dataset (used across all sections of this chapter)

📝 Note

Two values are intentionally missing: Sneha Pillai's salary and Meera Joshi's years of experience. These are shown as NaN (Not a Number) — pandas' way of representing missing data. We will address these in Section 2.

1 Data Selection and Filtering

When you work with a dataset that has dozens of columns and thousands of rows, you rarely need all of it at once. Data selection means pulling out only the columns you are interested in. Data filtering means keeping only the rows that satisfy a condition you define. Together, these two skills form the foundation of every data analysis task.

Data Selection	Choosing specific columns or rows from a DataFrame to work with, rather than using the entire table.

1.1 Selecting Columns

The simplest operation is to select one or more columns from a DataFrame. You do this using square brackets [ ] with the column name(s) inside.

Selecting a Single Column

When you select a single column, pandas returns a Series — a one-dimensional sequence of values with an index. Think of it as a single column from a spreadsheet.

```
import pandas as pd

# Load the employee dataset
df = pd.read_csv('employees.csv')

# Select a single column — returns a Series
names = df['name']
print(names)
```

Output:

```
0    Ananya Sharma
1      Rahul Verma
2       Priya Nair
3      Karan Mehta
4     Sneha Pillai
5       Vikram Das
6      Meera Joshi
7     Arjun Kapoor
Name: name, dtype: object
```

Figure 1.1 — Selecting the 'name' column returns a Series

Selecting Multiple Columns

When you need more than one column, pass a list of column names inside the square brackets. This returns a DataFrame (a table), not a Series.

```
# Select multiple columns — returns a DataFrame
# Note the double square brackets: outer [ ] for indexing, inner [ ] for the
list
subset = df[['name', 'department', 'salary']]
print(subset)
```

Output:

```
            name   department   salary
0  Ananya Sharma           HR  48000.0
1    Rahul Verma  Engineering  75000.0
2     Priya Nair      Finance  62000.0
3    Karan Mehta  Engineering  80000.0
4   Sneha Pillai           HR      NaN
5     Vikram Das      Finance  58000.0
6    Meera Joshi  Engineering  72000.0
7   Arjun Kapoor           HR  51000.0
```

Figure 1.2 — Selecting multiple columns returns a DataFrame

🔎 Why Double Square Brackets?

df['name'] — single brackets, selects ONE column, returns a Series.

df[['name', 'salary']] — double brackets, selects MULTIPLE columns, returns a DataFrame.

The outer [ ] is for indexing the DataFrame. The inner [ ] is a Python list of column names. This distinction is one of the most commonly confused aspects of pandas for beginners.

1.2 Selecting Rows — loc and iloc

While column selection uses column names, row selection works differently. pandas provides two dedicated tools for selecting rows: loc and iloc. Both are accessed using square brackets, but they work in fundamentally different ways.

loc (label-based)	iloc (position-based)
Selects rows using the index label	Selects rows using the integer position (0, 1, 2...)
Works like a name-tag lookup	Works like a position number lookup
End of range is inclusive	End of range is exclusive (like Python slicing)
Syntax: df.loc[row_label, col_name]	Syntax: df.iloc[row_position, col_position]

Table 1.1 — loc vs iloc: key differences

Using loc — Select by Label

loc selects rows using the row index label. In most datasets, the index is a sequential integer (0, 1, 2…). You can also select specific columns at the same time by providing them as a second argument.

```
# Select the row at index label 2 (Priya Nair)
print(df.loc[2])
```

Output:

```
emp_id                103
name          Priya Nair
department       Finance
salary           62000.0
experience             5
city             Chennai
Name: 2, dtype: object
```

Figure 1.3 — Selecting a single row using loc

```
# Select rows 1 through 3 AND only the 'name' and 'salary' columns
print(df.loc[1:3, ['name', 'salary']])
```

Output:

```
           name   salary
1  Rahul Verma  75000.0
2   Priya Nair  62000.0
3  Karan Mehta  80000.0
```

Figure 1.4 — Using loc to select a range of rows and specific columns

Using iloc — Select by Position

iloc selects rows using their integer position, starting from 0. This works exactly like list indexing in Python. Column positions are also integers, where 0 is the first column.

```
# Select the first row (position 0) — Ananya Sharma
print(df.iloc[0])

# Select rows at positions 0, 1, 2 and columns at positions 0 and 1
print(df.iloc[0:3, 0:2])
```

Output:

```
# First output — all columns for row 0:
emp_id    101
name    Ananya Sharma
...

# Second output — first 3 rows, first 2 columns:
   emp_id           name
0     101  Ananya Sharma
1     102    Rahul Verma
2     103     Priya Nair
```

Figure 1.5 — Using iloc to select by integer position

> ⚠️ **Important Difference: Inclusive vs Exclusive Range**
>
> df.loc[1:3] → includes rows 1, 2, AND 3 (end is inclusive)
>
> df.iloc[1:3] → includes rows 1 and 2 only (end is exclusive, like Python slicing)
>
> This difference is one of the most frequent sources of bugs in pandas code. Always double-check which accessor you are using and whether you need the end value included.

1.3 Filtering Rows Based on Conditions

Filtering means keeping only the rows that satisfy a specific condition. In pandas, you write a condition that evaluates to True or False for each row. Rows where the condition is True are kept; all others are dropped.

Single Condition Filter

To filter rows, write a comparison inside square brackets. For example, to keep only employees in the Engineering department:

```
# Filter: keep only Engineering department employees
engineers = df[df['department'] == 'Engineering']
print(engineers)
```

Output:

```
   emp_id         name   department   salary  experience       city
1     102  Rahul Verma  Engineering  75000.0         6.0  Bengaluru
3     104  Karan Mehta  Engineering  80000.0         8.0  Bengaluru
6     107  Meera Joshi  Engineering  72000.0         NaN       Pune
```

Figure 1.6 — Filtering rows where department equals 'Engineering'

```
# Filter: keep only employees with salary greater than 65000
high_earners = df[df['salary'] > 65000]
print(high_earners[['name', 'salary']])
```

Output:

```
          name   salary
1  Rahul Verma  75000.0
3  Karan Mehta  80000.0
6  Meera Joshi  72000.0
```

Figure 1.7 — Filtering rows where salary exceeds 65000

Multiple Condition Filter

You can combine multiple conditions using & (AND) and | (OR). Each condition must be wrapped in parentheses when combining.

```
# Filter: Engineering employees with salary > 70000
# Use & for AND, | for OR — each condition in parentheses
result = df[(df['department'] == 'Engineering') & (df['salary'] > 70000)]
print(result[['name', 'department', 'salary']])
```

Output:

```
          name   department   salary
1  Rahul Verma  Engineering  75000.0
3  Karan Mehta  Engineering  80000.0
6  Meera Joshi  Engineering  72000.0
```

Figure 1.8 — Combining two conditions with AND (&)

```
# Filter: employees from either Mumbai OR Chennai
mum_chn = df[(df['city'] == 'Mumbai') | (df['city'] == 'Chennai')]
print(mum_chn[['name', 'city']])
```

Output:

```
            name      city
0  Ananya Sharma    Mumbai
2     Priya Nair   Chennai
4   Sneha Pillai    Mumbai
7   Arjun Kapoor    Mumbai
```

Figure 1.9 — Filtering using OR condition

📝 Note — isin() for Multiple Values

When checking if a column's value belongs to a list of options, use isin() instead of multiple OR conditions. It is cleaner and easier to read.

```
df[df['city'].isin(['Mumbai', 'Chennai', 'Delhi'])]
```

This selects all rows where the city is Mumbai, Chennai, or Delhi. Much neater than writing three separate OR conditions.

2 Handling Missing Values

In the real world, datasets are almost never complete. Surveys have questions left unanswered. System exports have fields that were not recorded. Data merges leave gaps. These missing values — represented in pandas as NaN (Not a Number) — must be handled carefully before any analysis can produce reliable results.

Ignoring missing values can silently skew your results. For example, if you calculate the average salary and some salary values are missing, pandas will automatically exclude them — but you might not even realise this is happening unless you check first.

NaN	Not a Number — the value pandas uses to represent a missing or undefined entry in a DataFrame. It is a special floating-point value defined by the IEEE 754 standard.

2.1 Detecting Missing Values

Before you can fix missing values, you need to find them. pandas provides three essential tools for this.

isnull() — Find Missing Values

isnull() returns a DataFrame of the same shape as the original, but with True where a value is missing and False where it is present.

```
# Check for missing values across the entire DataFrame
print(df.isnull())
```

Output:

	emp_id	name	department	salary	experience	city
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	True	False	False
5	False	False	False	False	False	False
6	False	False	False	False	True	False
7	False	False	False	False	False	False

Figure 2.1 — isnull() returns a boolean DataFrame; True indicates a missing value

isnull().sum() — Count Missing Values Per Column

By chaining .sum() after isnull(), you get the total count of missing values in each column. This is far more useful in practice than the full True/False table.

```
# Count missing values per column
print(df.isnull().sum())
```

Output:

```
emp_id        0
name          0
department    0
salary        1
experience    1
city          0
dtype: int64
```

Figure 2.2 — Counting missing values in each column

💡 Pro Tip — Missing Value Percentage

For large datasets, knowing the count alone is not enough. Calculate the percentage instead:

```
print(df.isnull().sum() / len(df) * 100)
```

If a column has more than 50% missing values, you may need to decide whether to keep or drop that column entirely.

2.2 Dropping Missing Values

The simplest strategy for handling missing values is to remove the rows or columns that contain them. This is called dropping. Use dropna() for this purpose.

Dropping Rows with Any Missing Value

```
# Drop ALL rows that have at least one missing value
df_clean = df.dropna()
print(df_clean)
print('Rows remaining:', len(df_clean))
```

Output:

```
   emp_id           name   department   salary  experience       city
0     101  Ananya Sharma           HR  48000.0         3.0     Mumbai
1     102    Rahul Verma  Engineering  75000.0         6.0  Bengaluru
2     103     Priya Nair      Finance  62000.0         5.0    Chennai
3     104    Karan Mehta  Engineering  80000.0         8.0  Bengaluru
```

```
5     106     Vikram Das      Finance  58000.0         4.0      Delhi
7     108   Arjun Kapoor           HR  51000.0         3.0     Mumbai

Rows remaining: 6
```

Figure 2.3 — dropna() removes rows 4 and 6 (which had NaN values)

> ⚠️ **Be Careful with dropna()**
>
> dropna() permanently removes rows from the result. Do not reassign back to df unless you are certain you want to lose that data. It is safer to assign the result to a new variable like df_clean.
>
> Also, if your dataset has many columns and one column has frequent missing values, dropna() with no arguments will delete a lot of rows. Use the subset parameter to target only specific columns:
>
> df.dropna(subset=['salary']) # Only drop rows where 'salary' is missing

Dropping Columns with Missing Values

```
# Drop columns that have any missing value (use axis=1 for columns)
df_no_null_cols = df.dropna(axis=1)
print(df_no_null_cols.columns.tolist())
```

Output:

```
['emp_id', 'name', 'department', 'city']
```

Figure 2.4 — dropna(axis=1) drops 'salary' and 'experience' columns

2.3 Filling Missing Values

Rather than removing rows or columns, you can replace missing values with a sensible substitute. This is called imputation. The pandas method for this is fillna().

Fill with a Specific Value

```
# Fill missing salary with 0
df['salary'] = df['salary'].fillna(0)

# Fill missing experience with the string 'Unknown'
df['experience'] = df['experience'].fillna('Unknown')
```

```
print(df[['name', 'salary', 'experience']])
```

Output:

```
            name   salary experience
0  Ananya Sharma  48000.0          3
1    Rahul Verma  75000.0          6
2     Priya Nair  62000.0          5
3    Karan Mehta  80000.0          8
4   Sneha Pillai      0.0    Unknown
5     Vikram Das  58000.0          4
6    Meera Joshi  72000.0    Unknown
7   Arjun Kapoor  51000.0          3
```

Figure 2.5 — Missing values filled with 0 and 'Unknown'

Fill with Statistical Values (Mean, Median, Mode)

For numerical columns, filling with the mean or median of the existing values is a more intelligent strategy than using 0. This preserves the overall distribution of the data.

```
# Fill missing salary with the MEAN salary of all other employees
mean_salary = df['salary'].mean()
df['salary'] = df['salary'].fillna(mean_salary)

print('Mean salary used for fill:', round(mean_salary, 2))
print(df[['name', 'salary']])
```

Output:

```
Mean salary used for fill: 63714.29

            name    salary
0  Ananya Sharma  48000.00
1    Rahul Verma  75000.00
2     Priya Nair  62000.00
3    Karan Mehta  80000.00
4   Sneha Pillai  63714.29   <- was NaN, now filled with mean
5     Vikram Das  58000.00
6    Meera Joshi  72000.00
7   Arjun Kapoor  51000.00
```

Figure 2.6 — Filling missing salary with the column mean

🔎 Mean vs Median — When to Use Which?

Mean is the average of all values. It is sensitive to extreme outliers. If one employee earns 5,00,000 and the others earn 50,000, the mean will be much higher than what is typical.

Median is the middle value when all values are sorted. It is resistant to outliers. For salary data, median is often the better choice.
df['salary'].fillna(df['salary'].median())

3 Data Cleaning and Transformation

Data cleaning is the process of correcting or removing inaccurate, incomplete, or improperly formatted data from a dataset. Even when there are no missing values, data can still be 'dirty' — column values may have inconsistent capitalisation, extra spaces, wrong data types, or duplicate records. Transformation refers to reshaping or converting the data into the format your analysis requires.

These two tasks together — cleaning and transformation — typically consume the largest portion of a data analyst's working time. Getting this step right ensures that every subsequent analysis, visualisation, or model is built on reliable data.

3.1 Renaming Columns

Column names in raw data are often technical, abbreviated, or inconsistently cased. Renaming makes the DataFrame easier to work with and your code easier to read.

```
# Rename specific columns using a dictionary
# Old name : New name
df = df.rename(columns={
    'emp_id'     : 'Employee ID',
    'name'       : 'Full Name',
    'department' : 'Department',
    'salary'     : 'Salary (INR)',
    'experience' : 'Years of Experience',
    'city'       : 'City'
})

print(df.columns.tolist())
```

Output:

```
['Employee ID', 'Full Name', 'Department', 'Salary (INR)', 'Years of
Experience', 'City']
```

Figure 3.1 — Renaming columns using a dictionary

> 📝 **Note**
>
> rename() with a dictionary lets you rename only specific columns without affecting the rest. You do not have to list every column — only the ones you want to change.
>
> For the rest of Section 3, we revert to the original column names (emp_id, name, etc.) for consistency with the earlier sections.

3.2 Removing Duplicate Rows

Duplicate rows occur when the same record appears more than once. This can happen when data is collected from multiple sources and merged together, or due to system errors. Duplicates inflate counts and distort aggregations.

```
# Create a DataFrame that intentionally has a duplicate row
import pandas as pd

data = {
    'emp_id'    : [101, 102, 103, 102],
    'name'      : ['Ananya', 'Rahul', 'Priya', 'Rahul'],
    'department': ['HR', 'Engineering', 'Finance', 'Engineering']
}
df_dup = pd.DataFrame(data)

# Check for duplicates
print('Duplicate rows:', df_dup.duplicated().sum())

# Remove duplicate rows (keep the first occurrence)
df_no_dup = df_dup.drop_duplicates()
print(df_no_dup)
```

Output:

```
Duplicate rows: 1

   emp_id     name   department
0     101   Ananya           HR
1     102    Rahul  Engineering
2     103    Priya      Finance
```

Figure 3.2 — Detecting and removing duplicate rows

3.3 Changing Data Types

Each column in a DataFrame has a data type (dtype). pandas may not always infer the correct type when loading data. For example, a column of numbers stored as text cannot be used in arithmetic operations. You must convert it first using astype().

```
# Check data types of all columns
print(df.dtypes)
```

Output:

```
emp_id         int64
name          object
department    object
salary       float64
experience   float64
```

```
city          object
dtype: object
```

Figure 3.3 — Checking column data types using .dtypes

```
# Convert emp_id from int64 to string (useful if you don't want arithmetic on
IDs)
df['emp_id'] = df['emp_id'].astype(str)

# Convert experience from float to integer (remove the .0 decimal)
# First fill NaN, then convert
df['experience'] = df['experience'].fillna(0).astype(int)

print(df[['emp_id', 'experience']].dtypes)
```

Output:

```
emp_id       object
experience    int64
dtype: object
```

Figure 3.4 — Converting column data types using astype()

🔎 Common Data Types in pandas
int64 — Whole numbers (no decimal). E.g., employee ID, age, count
float64 — Numbers with decimals. E.g., salary, GPA, price
object — Text (strings). E.g., names, departments, city names
bool — True or False values
datetime64 — Dates and times. E.g., join date, transaction timestamp

3.4 Cleaning Text Data

Text columns often contain inconsistencies — extra spaces, mixed capitalisation, or formatting differences. pandas provides a .str accessor that lets you apply string operations directly to an entire column.

```
# Simulate a messy name column
df['name'] = ['  ananya sharma', 'RAHUL VERMA', 'Priya Nair ',
              'karan mehta', '  Sneha Pillai', 'vikram das',
              'Meera Joshi  ', 'arjun kapoor']

# Step 1: Remove leading and trailing whitespace
```

```
df['name'] = df['name'].str.strip()

# Step 2: Convert to Title Case (capitalise first letter of each word)
df['name'] = df['name'].str.title()

print(df['name'])
```

Output:

```
0    Ananya Sharma
1      Rahul Verma
2       Priya Nair
3      Karan Mehta
4     Sneha Pillai
5       Vikram Das
6      Meera Joshi
7     Arjun Kapoor
Name: name, dtype: object
```

Figure 3.5 — Cleaning text using .str.strip() and .str.title()

String Method	What It Does	Example
.str.strip()	Remove spaces from both ends	' hello ' → 'hello'
.str.upper()	Convert all text to uppercase	'hello' → 'HELLO'
.str.lower()	Convert all text to lowercase	'HELLO' → 'hello'
.str.title()	Capitalise first letter of each word	'hello world' → 'Hello World'
.str.replace(a,b)	Replace text a with text b	'Mr. Rahul' → 'Rahul'
.str.len()	Count the number of characters	'Alice' → 5

Table 3.1 — Commonly used pandas string methods

3.5 Creating New Columns

You can derive new columns from existing ones. For example, you might want to create a 'salary band' column based on the salary value, or a 'seniority' label based on years of experience. This is one of the most useful transformation techniques.

```
# Create a new column: annual bonus = 10% of salary
df['bonus'] = df['salary'] * 0.10

# Create a new column: seniority label based on experience
def get_seniority(exp):
    if exp <= 3:
        return 'Junior'
```

```
    elif exp <= 6:
        return 'Mid-Level'
    else:
        return 'Senior'

df['seniority'] = df['experience'].apply(get_seniority)

print(df[['name', 'salary', 'bonus', 'experience', 'seniority']])
```

Output:

```
            name    salary    bonus  experience  seniority
0  Ananya Sharma  48000.0   4800.0           3     Junior
1    Rahul Verma  75000.0   7500.0           6  Mid-Level
2     Priya Nair  62000.0   6200.0           5  Mid-Level
3    Karan Mehta  80000.0   8000.0           8     Senior
4   Sneha Pillai  48571.4   4857.1           2     Junior
5     Vikram Das  58000.0   5800.0           4  Mid-Level
6    Meera Joshi  72000.0   7200.0           0     Junior
7   Arjun Kapoor  51000.0   5100.0           3     Junior
```

Figure 3.6 — Creating new 'bonus' and 'seniority' columns from existing data

📝 Note — The apply() Method

apply() lets you apply a custom function to every value in a column. You define a function (like get_seniority above), then pass it to apply() — pandas automatically calls your function once for each row's value and builds a new column from the results.

apply() is extremely powerful. Whenever you need to transform a column using logic that is too complex for a simple arithmetic expression, apply() is the right tool.

4 Sorting and Ranking Data

Sorting reorganises the rows of a DataFrame based on the values in one or more columns. Ranking assigns a numerical rank to each row based on its relative position within a column's values. Both are essential for spotting the highest or lowest values, creating leaderboards, identifying outliers, and preparing data for reports.

4.1 Sorting by a Single Column

Use sort_values() to sort a DataFrame by the values in a specific column. By default, it sorts in ascending order (smallest to largest).

```
# Sort employees by salary — ascending order (lowest to highest)
df_sorted = df.sort_values('salary')
print(df_sorted[['name', 'salary']])
```

Output:

```
            name    salary
0  Ananya Sharma  48000.0
7   Arjun Kapoor  51000.0
5     Vikram Das  58000.0
2     Priya Nair  62000.0
6    Meera Joshi  72000.0
1    Rahul Verma  75000.0
3    Karan Mehta  80000.0
4   Sneha Pillai      NaN
```

Figure 4.1 — Sorting by salary in ascending order; NaN values appear at the bottom by default

```
# Sort by salary — descending order (highest to lowest)
df_sorted_desc = df.sort_values('salary', ascending=False)
print(df_sorted_desc[['name', 'salary']])
```

Output:

```
            name    salary
3    Karan Mehta  80000.0
1    Rahul Verma  75000.0
6    Meera Joshi  72000.0
2     Priya Nair  62000.0
5     Vikram Das  58000.0
7   Arjun Kapoor  51000.0
0  Ananya Sharma  48000.0
4   Sneha Pillai      NaN
```

Figure 4.2 — Sorting by salary in descending order

4.2 Sorting by Multiple Columns

You can sort by more than one column by passing a list. pandas will sort by the first column, and if two rows have the same value, it will use the second column to break the tie.

```
# Sort by department (A-Z), then by salary within each department (high to low)
df_multi = df.sort_values(
    by=['department', 'salary'],
    ascending=[True, False]
)
print(df_multi[['name', 'department', 'salary']])
```

Output:

```
            name   department   salary
3    Karan Mehta  Engineering  80000.0
1    Rahul Verma  Engineering  75000.0
6    Meera Joshi  Engineering  72000.0
2     Priya Nair      Finance  62000.0
5     Vikram Das      Finance  58000.0
7   Arjun Kapoor           HR  51000.0
0  Ananya Sharma           HR  48000.0
4   Sneha Pillai           HR      NaN
```

Figure 4.3 — Multi-column sort: by department alphabetically, then by salary descending within each department

4.3 Ranking Values

rank() assigns a numerical rank to each value in a column. The smallest value gets rank 1, the next gets rank 2, and so on. Ranking is useful for creating performance leaderboards, scoring tables, and comparative reports.

```
# Add a salary rank column (rank 1 = highest salary)
# ascending=False means the highest salary gets rank 1
df['salary_rank'] = df['salary'].rank(ascending=False)

print(df[['name', 'salary', 'salary_rank']].sort_values('salary_rank'))
```

Output:

```
            name   salary  salary_rank
3    Karan Mehta  80000.0          1.0
1    Rahul Verma  75000.0          2.0
6    Meera Joshi  72000.0          3.0
2     Priya Nair  62000.0          4.0
5     Vikram Das  58000.0          5.0
7   Arjun Kapoor  51000.0          6.0
0  Ananya Sharma  48000.0          7.0
4   Sneha Pillai      NaN          NaN
```

Figure 4.4 — Ranking employees by salary; NaN values receive no rank

🔎 **Handling Ties in Ranking**
When two rows have the same value, how do they share a rank? pandas offers several methods:
df['col'].rank(method='average') → tied values share the average of their ranks (default)
df['col'].rank(method='min') → tied values both get the lower rank
df['col'].rank(method='dense') → no gaps in ranking; tied values get the same rank
df['col'].rank(method='first') → earlier row in the DataFrame gets the lower rank

5 Grouping and Aggregation (groupby())

Grouping is one of the most powerful features in pandas. The concept is simple: split the data into groups based on the values in one or more columns, apply a calculation to each group separately, and combine the results back into a single table. This is known as the Split-Apply-Combine pattern.

For example, instead of knowing the salary of each individual employee, you might want to know the average salary per department. That is exactly what groupby() does — it groups all employees by their department and lets you apply an aggregation function to each group.

5.1 Basic groupby() with Aggregation

```
# Group by department and compute the mean salary for each department
dept_salary = df.groupby('department')['salary'].mean()
print(dept_salary)
```

Output:

```
department
Engineering    75666.67
Finance        60000.00
HR             49500.00
Name: salary, dtype: float64
```

Figure 5.1 — Average salary per department using groupby()

```
# Count the number of employees in each department
dept_count = df.groupby('department')['emp_id'].count()
print(dept_count)
```

Output:

```
department
Engineering    3
Finance        2
HR             3
Name: emp_id, dtype: int64
```

Figure 5.2 — Employee count per department

5.2 Multiple Aggregations with agg()

agg() allows you to apply multiple aggregation functions at the same time. Instead of running groupby() separately for mean, max, and count, you can get all three in a single operation.

```
# Get multiple statistics for salary, grouped by department
dept_stats = df.groupby('department')['salary'].agg(['mean', 'max', 'min',
'count'])
print(dept_stats)
```

Output:

```
                mean    max     min  count
department
Engineering  75666.67  80000.0  72000.0      3
Finance      60000.00  62000.0  58000.0      2
HR           49500.00  51000.0  48000.0      2
```

Figure 5.3 — Multiple aggregations on salary grouped by department

```
# Apply different aggregations to different columns
result = df.groupby('department').agg({
    'salary'    : ['mean', 'max'],
    'experience': 'mean',
    'emp_id'    : 'count'
})
print(result)
```

Output:

```
               salary          experience emp_id
                 mean      max       mean  count
department
Engineering  75666.67  80000.0       7.00      3
Finance      60000.00  62000.0       4.50      2
HR           49500.00  51000.0       2.67      3
```

Figure 5.4 — Applying different aggregations to different columns

5.3 Grouping by Multiple Columns

You can group by more than one column to get more granular summaries. For example, grouping by both department and city tells you the average salary by department and city combination.

```
# Group by department AND city
dept_city = df.groupby(['department', 'city'])['salary'].mean().reset_index()
print(dept_city)
```

Output:

```
   department       city    salary
0  Engineering  Bengaluru  77500.0
1  Engineering       Pune  72000.0
2      Finance    Chennai  62000.0
3      Finance      Delhi  58000.0
```

```
4            HR      Mumbai   49500.0
```

Figure 5.5 — Grouping by two columns to compute average salary by department and city

📝 Note — reset_index()

After a groupby() operation, the grouped columns become the index. This can make the result awkward to work with. Calling .reset_index() converts the index back into regular columns, giving you a clean flat DataFrame that is easier to filter, sort, and display.

6 Merging and Joining DataFrames

In real projects, data rarely comes from a single source. You might have one table with employee details and another with their department budgets. To analyse them together, you need to combine them. pandas provides two main methods for this: merge() and concat().

This concept is directly analogous to SQL JOIN operations and is one of the most important skills in data manipulation.

6.1 Understanding Merge Types

Before writing any code, it is important to understand the four types of merges. Each answers a different question about which rows to keep.

Merge Type	Rows Kept	Use When You Want To
inner	Only rows that match in BOTH tables	Find records present in both datasets
left	All rows from the LEFT table; matched rows from right	Keep all left records, add right data where available
right	All rows from the RIGHT table; matched rows from left	Keep all right records, add left data where available
outer	All rows from BOTH tables	Keep everything, with NaN where no match exists

Table 6.1 — The four types of DataFrame merges

6.2 Inner Merge — Most Common

An inner merge keeps only the rows that have a matching key value in both DataFrames. Rows with no match are discarded.

```
import pandas as pd

# Employee DataFrame (left)
employees = pd.DataFrame({
    'emp_id'    : [101, 102, 103, 104, 105],
    'name'      : ['Ananya', 'Rahul', 'Priya', 'Karan', 'Sneha'],
    'dept_id'   : [10, 20, 30, 20, 10]
})

# Department DataFrame (right)
departments = pd.DataFrame({
    'dept_id'   : [10, 20, 30, 40],
    'dept_name' : ['HR', 'Engineering', 'Finance', 'Marketing'],
```

```
    'budget'    : [500000, 1500000, 800000, 600000]
})

# Inner merge — only employees whose dept_id exists in both tables
merged = pd.merge(employees, departments, on='dept_id', how='inner')
print(merged)
```

Output:

```
   emp_id     name  dept_id     dept_name    budget
0     101   Ananya       10            HR    500000
1     102    Rahul       20   Engineering   1500000
2     103    Priya       30       Finance    800000
3     104    Karan       20   Engineering   1500000
4     105    Sneha       10            HR    500000
```

Figure 6.1 — Inner merge combines employee and department data via dept_id

6.3 Left Merge — Preserve All Left Records

A left merge keeps all rows from the left DataFrame and adds matching data from the right. If no match is found, NaN fills the right-side columns.

```
# Add employee 106 with dept_id 99 (which does NOT exist in departments table)
extra = pd.DataFrame({'emp_id':[106], 'name':['Vikram'], 'dept_id':[99]})
employees_ext = pd.concat([employees, extra])

# Left merge — keeps ALL employees, even those with no matching department
left_merged = pd.merge(employees_ext, departments, on='dept_id', how='left')
print(left_merged)
```

Output:

```
   emp_id     name  dept_id     dept_name      budget
0     101   Ananya       10            HR  500000.0
1     102    Rahul       20   Engineering  1500000.0
2     103    Priya       30       Finance   800000.0
3     104    Karan       20   Engineering  1500000.0
4     105    Sneha       10            HR   500000.0
5     106   Vikram       99           NaN        NaN  <- no match, NaN added
```

Figure 6.2 — Left merge preserves all 6 employees; Vikram has NaN for department info since dept_id 99 does not exist

🔎 Choosing the Right Merge Type

Inner merge — Use when you only want records that exist in both tables. You will lose unmatched rows from both sides.

Left merge — Use when the left table is your 'main' table and you want to enrich it with data from the right table. Unmatched left rows are always preserved.
Outer merge — Use when you cannot afford to lose any records from either table and are willing to deal with NaN values in the result.

6.4 Concatenating DataFrames with concat()

While merge() combines DataFrames horizontally (adding columns), concat() stacks DataFrames vertically (adding rows). Use concat() when you have multiple DataFrames with the same columns and you want to combine them into one.

```
# Two batches of employee data from different cities
batch_mumbai = pd.DataFrame({
    'emp_id'     : [201, 202],
    'name'       : ['Rohan Iyer', 'Neha Gupta'],
    'city'       : ['Mumbai', 'Mumbai']
})

batch_delhi = pd.DataFrame({
    'emp_id'     : [301, 302],
    'name'       : ['Amit Patel', 'Divya Singh'],
    'city'       : ['Delhi', 'Delhi']
})

# Stack them vertically
all_employees = pd.concat([batch_mumbai, batch_delhi], ignore_index=True)
print(all_employees)
```

Output:

```
   emp_id         name    city
0     201   Rohan Iyer  Mumbai
1     202   Neha Gupta  Mumbai
2     301   Amit Patel   Delhi
3     302  Divya Singh   Delhi
```

Figure 6.3 — concat() stacks two DataFrames vertically; ignore_index=True resets the index

📝 Note — ignore_index=True

When you concat() two DataFrames, their original row indices are carried over. Without ignore_index=True, the result would show index values 0, 1, 0, 1 — repeating. Setting ignore_index=True resets the index to 0, 1, 2, 3 — a clean continuous sequence.

ETHANS
CLOUD & AI ACADEMY

7 Basic Data Analysis and Summary Statistics

After selecting, cleaning, sorting, grouping, and merging your data, you are ready for the final step — extracting meaningful insights through analysis. Summary statistics summarise the key numerical properties of your data in a compact, readable form. They answer fundamental questions: What is the average? What is the range? How spread out are the values?

pandas makes this remarkably easy. With just a few lines of code, you can get a comprehensive picture of your dataset.

7.1 The describe() Method — One-Line Summary

describe() is the fastest way to get a statistical overview of all numerical columns in your DataFrame. It computes eight statistics at once: count, mean, standard deviation, minimum, 25th percentile, 50th percentile (median), 75th percentile, and maximum.

```
# Get a full statistical summary of all numeric columns
print(df.describe())
```

Output:

```
             salary  experience
count      7.000000    7.000000
mean   63714.285714    4.142857
std    12133.565487    1.951499
min    48000.000000    2.000000
25%    54500.000000    3.000000
50%    62000.000000    4.000000
75%    73500.000000    5.500000
max    80000.000000    8.000000
```

Figure 7.1 — describe() provides 8 key statistics for every numeric column. Count is 7 because one salary value was NaN.

Statistic	What It Tells You	In Our Dataset (salary)
count	Number of non-null values used in the calculation	7 (one NaN excluded)
mean	Arithmetic average of all values	63,714 INR
std	Standard deviation — how spread out values are from the mean	12,134 INR
min	Smallest value in the column	48,000 INR (Ananya)
25%	25th percentile — 25% of values fall below this	54,500 INR
50%	Median — the exact middle value	62,000 INR

Statistic	What It Tells You	In Our Dataset (salary)
75%	75th percentile — 75% of values fall below this	73,500 INR
max	Largest value in the column	80,000 INR (Karan)

Table 7.1 — Understanding the output of describe() using the salary column

7.2 Individual Statistical Functions

You do not always need the full describe() output. pandas provides individual functions for each statistic, which you can apply to any column.

```
print('Mean salary:    ', df['salary'].mean())
print('Median salary:  ', df['salary'].median())
print('Std deviation:  ', df['salary'].std())
print('Minimum salary: ', df['salary'].min())
print('Maximum salary: ', df['salary'].max())
print('Total salary bill:', df['salary'].sum())
print('Employees with data:', df['salary'].count())
```

Output:

```
Mean salary:     63714.285714...
Median salary:   62000.0
Std deviation:   12133.565487...
Minimum salary:  48000.0
Maximum salary:  80000.0
Total salary bill: 446000.0
Employees with data: 7
```

Figure 7.2 — Individual statistical functions applied to the salary column

🔎 Mean vs Median — Which to Use for Salary?

The mean salary is 63,714 INR. The median is 62,000 INR. These are close but not identical.

If you added one very high-earning employee (say, 5,00,000 INR), the mean would jump significantly while the median would barely change. For income and salary data, the median is generally considered a more honest representation of a 'typical' employee's pay.

Use mean when data is symmetric and has no extreme outliers. Use median when data may be skewed.

7.3 Value Counts — Frequency Distribution

value_counts() tells you how many times each unique value appears in a column. It is most useful for categorical columns like department or city.

```
# How many employees are in each department?
print(df['department'].value_counts())
```

Output:

```
department
HR             3
Engineering    3
Finance        2
Name: count, dtype: int64
```

Figure 7.3 — value_counts() shows the frequency of each unique department value

```
# Show as percentage instead of count
print(df['department'].value_counts(normalize=True) * 100)
```

Output:

```
department
HR             37.5
Engineering    37.5
Finance        25.0
Name: proportion, dtype: float64
```

Figure 7.4 — value_counts(normalize=True) shows each category as a percentage of total records

7.4 Correlation Analysis

Correlation measures how strongly two numerical variables move together. A value close to +1 means both increase together (positive correlation). A value close to -1 means one increases as the other decreases. A value near 0 means no relationship.

```
# Compute pairwise correlation between all numeric columns
print(df[['salary', 'experience']].corr())
```

Output:

```
              salary  experience
salary      1.000000    0.887412
experience  0.887412    1.000000
```

Figure 7.5 — Correlation matrix showing a strong positive relationship (0.887) between salary and experience

📝 Interpreting This Result

The correlation between salary and experience is 0.887. This is close to 1, which means there is a strong positive relationship: employees with more years of experience tend to earn higher salaries. This makes intuitive sense in our dataset.

The diagonal values are always 1.0 — a column is perfectly correlated with itself.

7.5 Putting It All Together — A Mini Analysis

Let us bring together the techniques from all seven sections to produce a complete, meaningful analysis of our employee dataset in just a few lines of code.

```
import pandas as pd

df = pd.read_csv('employees.csv')

# Step 1: Check data quality
print('Missing values:')
print(df.isnull().sum())

# Step 2: Fill missing salary with median
df['salary'] = df['salary'].fillna(df['salary'].median())
df['experience'] = df['experience'].fillna(df['experience'].median())

# Step 3: Create a seniority column
df['seniority'] = df['experience'].apply(
    lambda x: 'Senior' if x > 5 else ('Mid-Level' if x > 3 else 'Junior')
)

# Step 4: Department-level summary
dept_summary = df.groupby('department').agg(
    total_employees = ('emp_id', 'count'),
    avg_salary      = ('salary', 'mean'),
    max_salary      = ('salary', 'max'),
    avg_experience  = ('experience', 'mean')
).round(2)

print('\nDepartment Summary:')
print(dept_summary)
```

Output:

```
Missing values:
salary        1
experience    1
(all other columns: 0)
```

```
Department Summary:
             total_employees  avg_salary  max_salary  avg_experience
department
Engineering                3    75666.67     80000.0            7.00
Finance                    2    60000.00     62000.0            4.50
HR                         3    49500.00     51000.0            2.67
```

Figure 7.6 — A complete analysis pipeline from raw data to department-level insights

🔎 What This Analysis Tells Us

Engineering has the highest average salary (75,667 INR) and the most experienced team (7 years average).

HR has the most employees (3) but the lowest average salary (49,500 INR) and least experience (2.67 years average).

Finance sits in the middle on all metrics — 2 employees, average salary 60,000 INR, 4.5 years average experience.

This type of structured summary is exactly what a manager or business analyst would want to see before making staffing or budget decisions.

Chapter Summary

This chapter covered the seven core techniques of data handling with pandas. Here is a concise recap of everything you learned:

Section	Topic	Key Method(s)
1	Data Selection and Filtering	df['col'], df[['c1','c2']], .loc[], .iloc[], boolean filter
2	Handling Missing Values	.isnull(), .dropna(), .fillna()
3	Data Cleaning and Transformation	.rename(), .drop_duplicates(), .astype(), .str., .apply()
4	Sorting and Ranking Data	.sort_values(), .rank()
5	Grouping and Aggregation	.groupby(), .agg(), .reset_index()
6	Merging and Joining	pd.merge(), pd.concat()
7	Summary Statistics	.describe(), .mean(), .median(), .value_counts(), .corr()

Table S.1 — Chapter 2 Summary Reference

- Data selection picks specific rows and columns using loc (label-based) or iloc (position-based). Filtering uses boolean conditions to keep only the rows you need.
- Missing values are represented as NaN. Always check for them with isnull().sum() before any analysis. Drop them with dropna() or fill them with fillna() using a constant, mean, or median.
- Data cleaning corrects dirty data: rename columns, remove duplicates, fix data types, and standardise text. Use apply() to derive new columns from custom logic.
- Sorting uses sort_values() and accepts multiple columns and ascending/descending options. rank() assigns a numerical rank to each value in a column.
- groupby() implements the Split-Apply-Combine pattern — split data into groups, apply an aggregation function, and combine results. agg() allows multiple functions in one call.
- merge() combines DataFrames horizontally using a common key. The four merge types (inner, left, right, outer) control which rows are kept. concat() stacks DataFrames vertically.
- describe() provides an 8-statistic summary of all numeric columns in one call. value_counts() shows the frequency distribution of categorical values. corr() measures the relationship between numeric variables.